

ORDENAÇÃO E PESQUISA

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Introdução a Ordenação
- Métodos de Ordenação
- QuickSort
- Introdução a Pesquisa
- Pesquisa Binária
- Conclusão
- Dúvidas

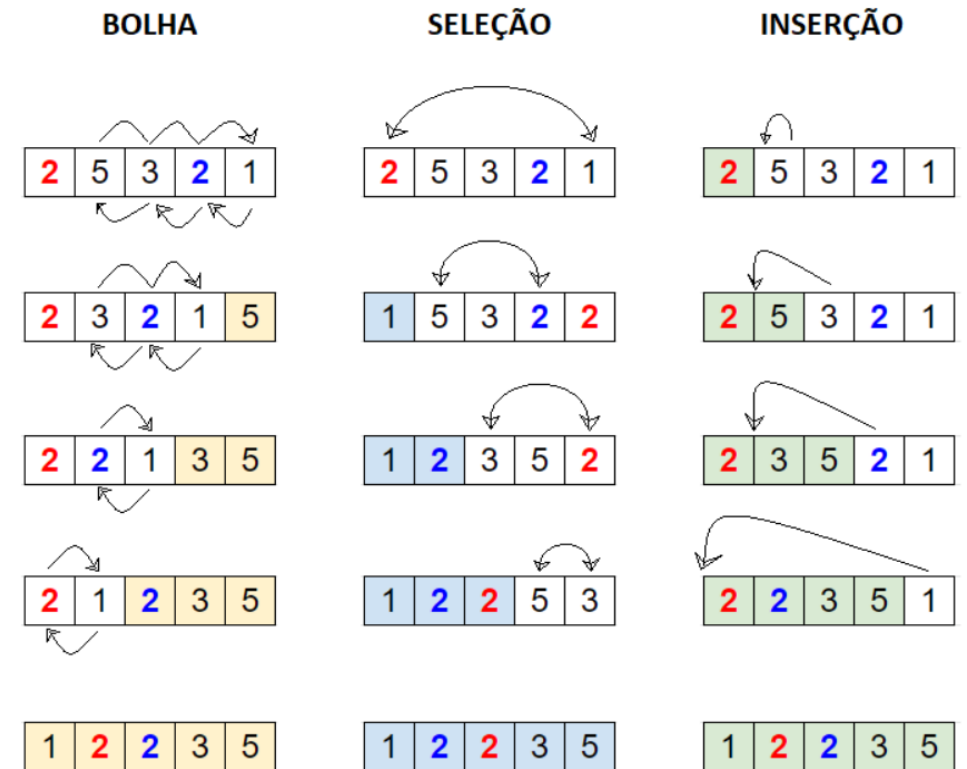


INTRODUÇÃO

- **Ordenação** é o processo de organizar elementos de uma coleção (como vetores, matrizes, listas ou registros) de acordo com uma **ordem específica**, geralmente crescente ou decrescente.
- Importância da ordenação:
 - Eficiência em busca de dados
 - Melhora na organização e legibilidade:
 - Base para algoritmos avançados - árvores binárias, heaps e tabelas hash
 - Otimização de desempenho

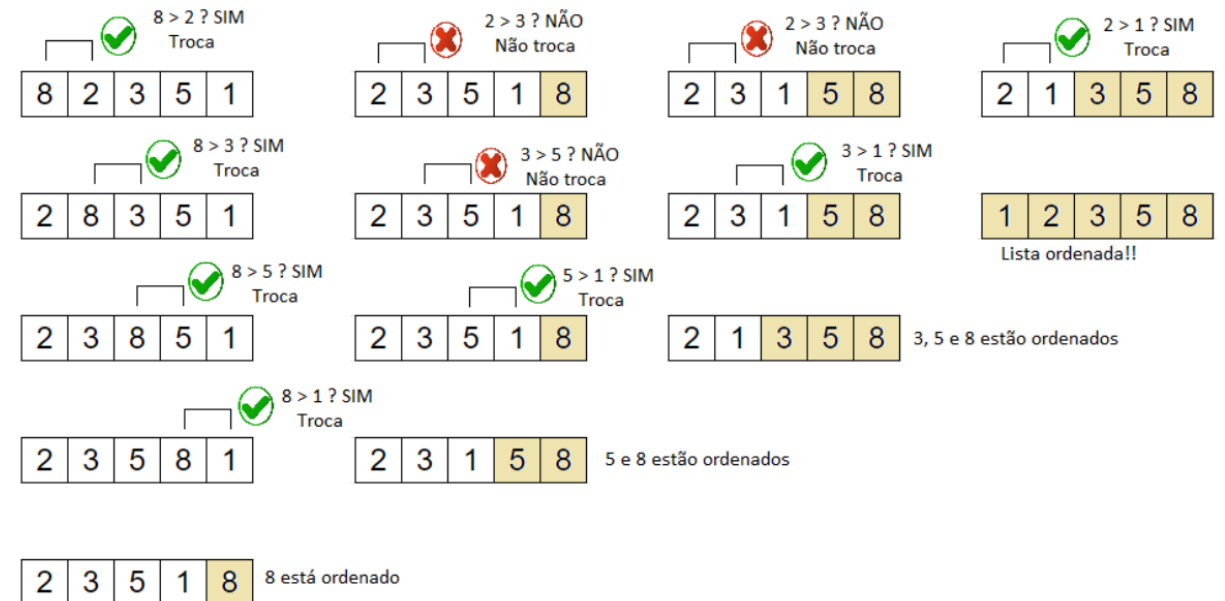
MÉTODOS DE ORDENAÇÃO

- **Métodos simples:** Bolha, Seleção, Inserção
 - São didáticos
 - Bons para listas pequenas
 - Lentidão em listas grandes
- **Método eficiente:** QuickSort
 - Criado por Tony Hoare em 1959
 - Eficientes para grandes volumes de dados
 - Usado amplamente



ORDENAÇÃO BOLHA

- **Bolha:** funciona percorrendo a lista, comparando elementos adjacentes (pares consecutivos) e trocando-os de posição caso não estejam em ordem.
- A cada iteração, o **maior** elemento “flutua” para a última posição, e o processo é repetido até que todos os elementos estejam na ordem correta.



ORDENAÇÃO BOLHA

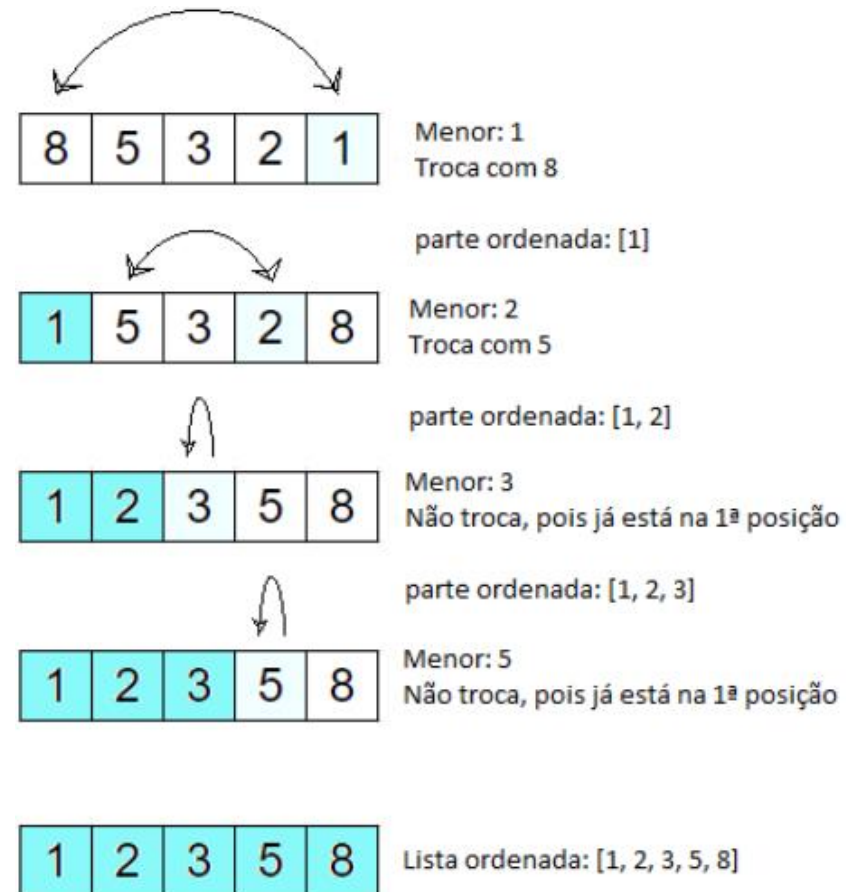
- **Exemplo:**

```
void bolha (int n, int* v);  
int main () {  
    int i;  
    int v[8] = {25, 48, 37, 12, 57, 86, 33, 92};  
    bolha(8, v);  
    printf("Vetor ordenado: ");  
    for (i=0; i<8; i++)  
        printf("%d ", v[i]);  
    return 0;  
}
```

```
void bolha (int n, int* v)  
{  
    int i, j;  
    for (i=n-1; i>=1; i--)  
        for (j=0; j<i; j++)  
            if (v[j]>v[j+1]) { //troca  
                int temp = v[j];  
                v[j] = v[j+1];  
                v[j+1] = temp;  
            }  
}
```

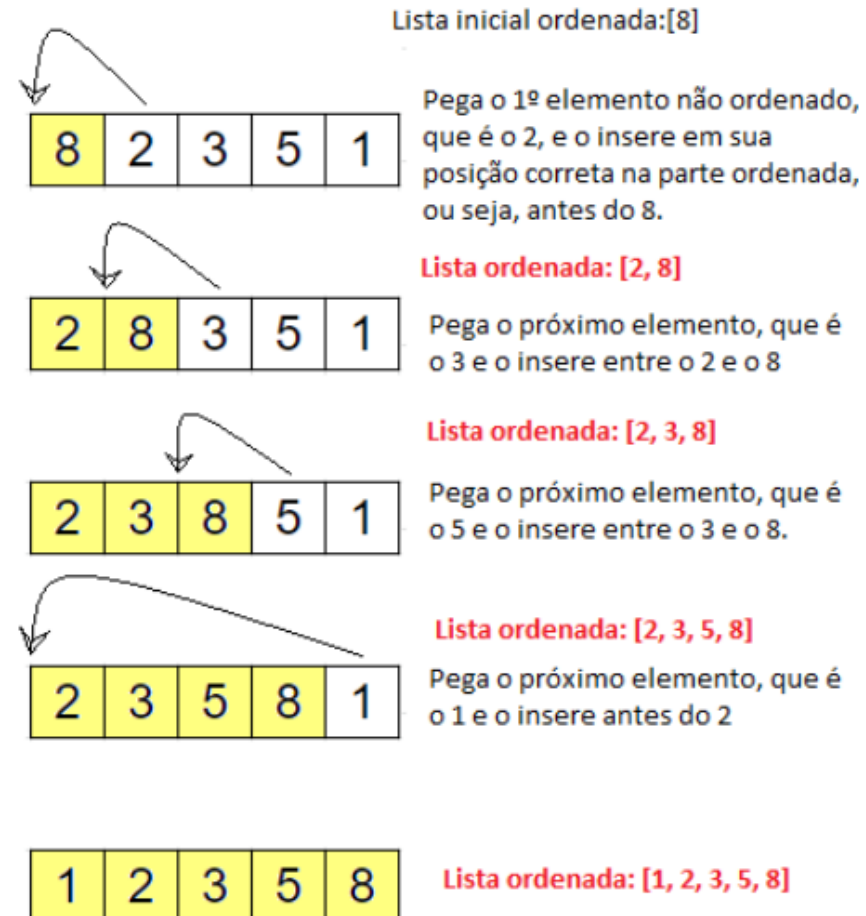
ORDENAÇÃO POR SELEÇÃO

- **Seleção:** funciona selecionando o **menor** elemento de uma lista não ordenada e colocando-o na primeira posição.
- Esse processo é realizado a cada iteração, excluindo os elementos já ordenados, ou seja, sempre selecionando o menor elemento da lista remanescente, que não está ordenada, e colocando na primeira posição dela.



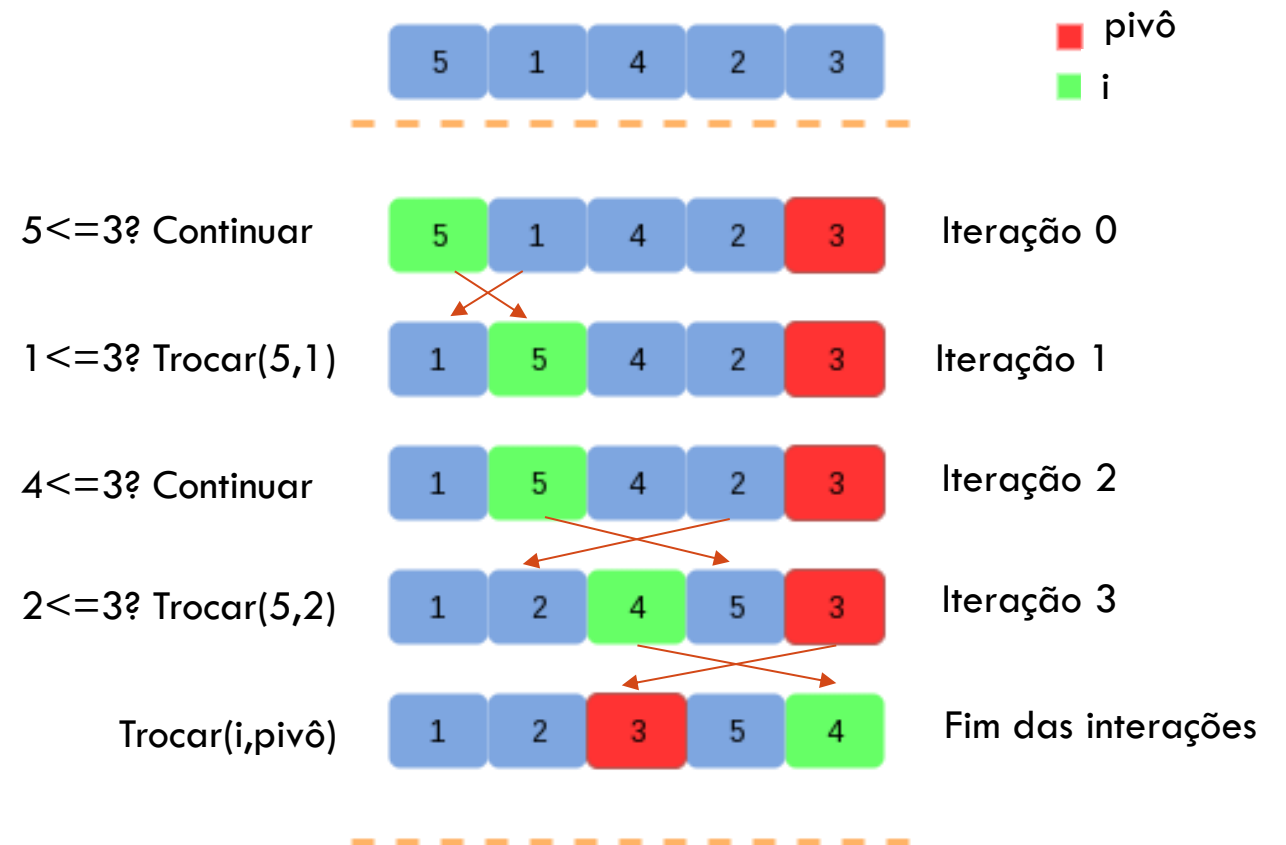
ORDENAÇÃO POR INSERÇÃO

- **Inserção:** funciona **dividindo a lista em duas partes**, uma com elementos já ordenados e a outra com elementos ainda não ordenados.
- A cada iteração do algoritmo, o primeiro elemento da parte não ordenada é inserido em sua posição correta na parte ordenada. Ao final das iterações, a lista estará ordenada.



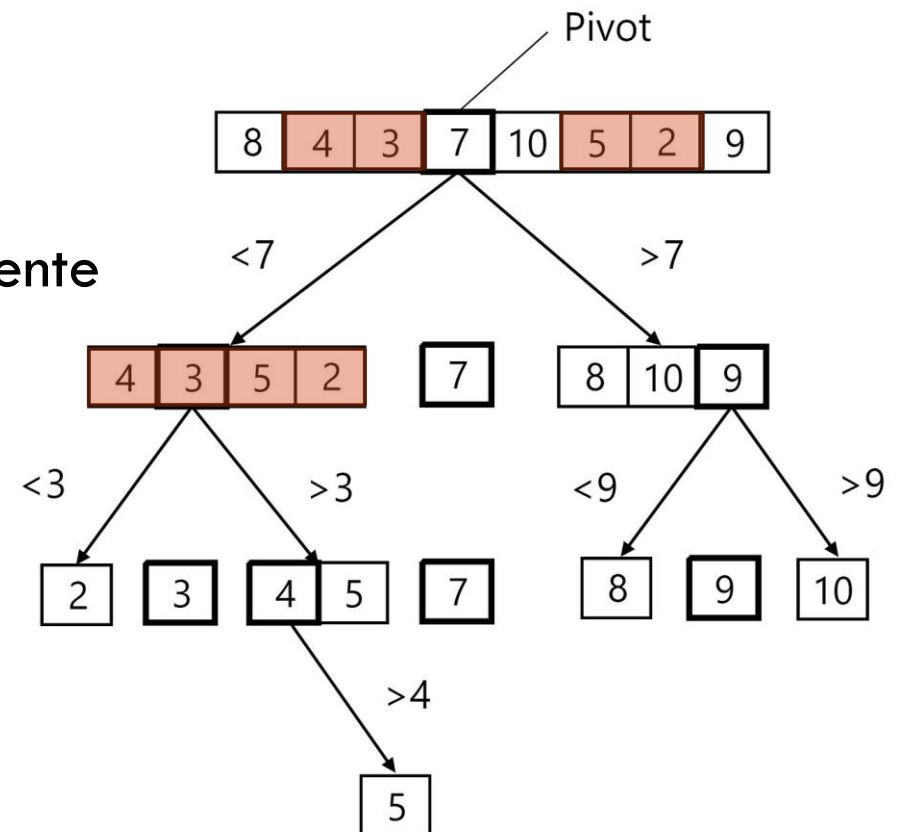
QUICKSORT

- Estratégia "Dividir e Conquistar"
- Escolhe um elemento pivô
- Particiona os elementos:
 - Menores que o pivô à esquerda
 - Maiores que o pivô à direita
- Repete o processo recursivamente



QUICKSORT

- Exemplo:
- Pivô localizado em uma posição diferente
- Na sequência é aplicado o algoritmo novamente



QUICKSORT

- Função `qsort()` na biblioteca padrão (`stdlib.h`)
- **Protótipo:** `void qsort(void *base, size_t num, size_t size, int (*compar)(const void*, const void*));`
- **Parâmetros:**
 - `base`: endereço do vetor a ser ordenado
 - `num`: número de elementos do vetor
 - `size`: tamanho de cada elemento
 - `compar`: ponteiro para função de comparação

QUICKSORT

- Exemplo simples com `qsort()` ordenando um vetor de inteiros:

```
#include <stdlib.h>

int comp(const void *a, const void *b);

int main() {
    int i, valores[10] = {2, 5, 1, 9, 4, 7, 3, 8, 6, 0};
    qsort(valores, 10, sizeof(int), comp);
    for(i=0; i<10; i++)
        printf("%i\n", valores[i]);
    return 0;
}

int comp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
```

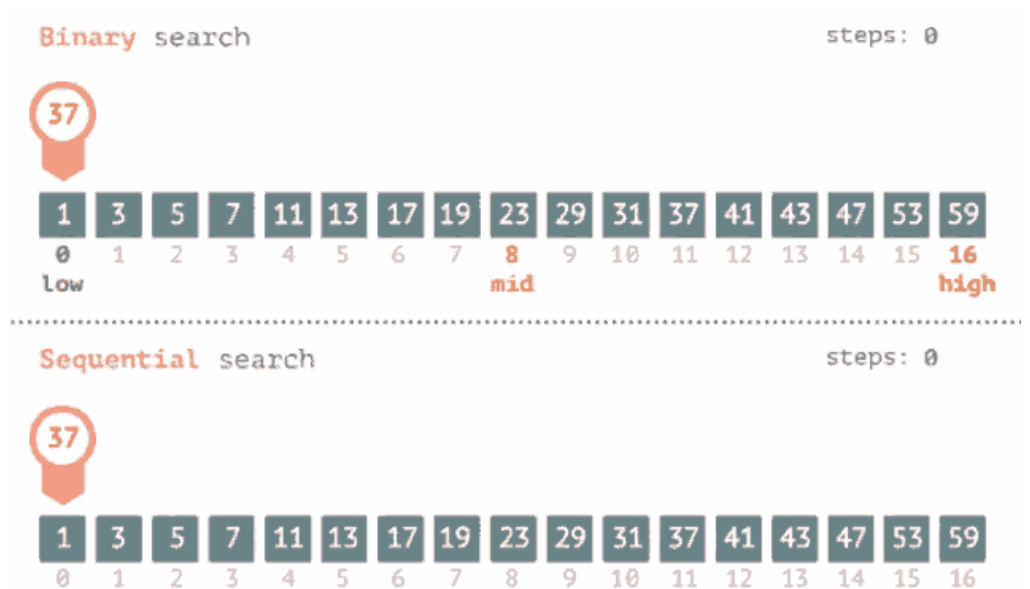
PESQUISA



- **Pesquisa** é o processo de processo de localizar um **elemento específico** dentro de uma coleção de dados.
- Importância da pesquisa:
 - Acesso rápido a informações: encontra registros específicos em grandes bases de dados.
 - Base de operações complexas: pesquisa rápida para sistemas de recomendação, bancos de dados e motores de busca.
 - Redução de tempo computacional: é possível localizar informações muito mais rápido do que com busca sequencial.
- Exemplos de algoritmos de pesquisa:
 - Busca sequencial (*Linear Search*)
 - Busca binária (*Binary Search*, requer dados ordenados)
 - Pesquisa em estruturas complexas (árvores, grafos, tabelas hash)

PESQUISA BINÁRIA

- Pesquisa eficiente em listas ordenadas
- Estratégia:
 - Verifica o elemento do meio
 - Se for menor, pesquisa na metade esquerda
 - Se for maior, pesquisa na metade direita
 - Repete até encontrar ou esgotar os elementos
- Complexidade: $O(\log n)$



PESQUISA BINÁRIA

- Função `bsearch()` na biblioteca padrão (`stdlib.h`)
- **Protótipo:**

```
void *bsearch(const void *key, const void *base,  
             size_t num, size_t size,  
             int (*compar)(const void*, const void*));
```
- **Parâmetros:**
 - `base`: endereço do vetor
 - `num`: número de elementos
 - `size`: tamanho de cada elemento
 - `compar`: ponteiro para função de comparação
- **Retorno:** ponteiro para o elemento encontrado e `NULL` se não encontrado

PESQUISA BINÁRIA

- Função `bsearch()` na biblioteca padrão (`stdlib.h`)
- **Protótipo:**

```
void *bsearch(const void *key, const void *base,  
             size_t num, size_t size,  
             int (*compar)(const void*, const void*));
```

- **Exemplo:**

```
int valores[8] = {1, 3, 5, 7, 9, 11, 13, 15};  
int chave = 7;  
int *resultado;  
resultado = (int*) bsearch(&chave, valores, 8, sizeof(int), comp);  
if(resultado != NULL)  
    printf("Valor encontrado!\n");  
else  
    printf("Valor não encontrado.\n");
```

QUICKSORT E PESQUISA BINÁRIA

- Exemplo que combina as duas funções:
 - Ordena o vetor com `qsort()`
 - Pesquisa com `bsearch()`
- Exemplo:

```
int resultado, busc;  
  
int valores[10] = {2, 5, 1, 9, 4, 7, 3, 8, 6, 0};  
  
qsort(valores, 10, sizeof(int), comp);  
  
scanf("%i", &busc);  
  
resultado = (int*)bsearch(&busc, valores, 10, sizeof(int), comp);
```

QUICKSORT E PESQUISA BINÁRIA

- Exemplo que compara estrutura usando o `qsort()`:
- Comparação por nome ou número:
- **Exemplo:** `struct aluno {int matricula; char nome[50];};`
- A função `comp`:

```
int comp(const void *a, const void *b) {  
    return strcmp(((struct aluno*)a)->nome,  
                  ((struct aluno*)b)->nome);  
}
```

CONCLUSÕES

- **Ordenação:**
- QuickSort é rápido e eficiente
- `qsort()` implementa QuickSort
- **Pesquisa:**
- Pesquisa binária exige vetor ordenado
- `bsearch()` facilita a busca
- Usando algoritmos eficientes (ex.: busca binária em dados ordenados), é possível localizar informações muito mais rápido do que com busca sequencial.

EXERCÍCIOS

- **Ex1** - Ordenar 10 números e mostrar antes/depois.
 - Versão 1: utilizando a ordenação bolha.
 - Versão 2: utilizando a função `qsort()`.
- **Ex2** - Ordenar 10 palavras e pesquisar por uma palavra específica.
 - Versão 1: utilizando a ordenação bolha e busca sequencial.
 - Versão 2: utilizando as funções `qsort()` e `bsearch()`.



EXERCÍCIOS

- **Ex3** - Definir a struct de aluno, ordenando por nota e pesquisando por nome

```
typedef struct Aluno{  
    char nome[50];  
    float nota;  
} Aluno;
```



DÚVIDAS

- Questionamentos?

